

Performance Benchmarking Report: NYC Taxi Tipping Performance Index (TPI)

Student Name: Yi Kuang

1. Methodology & Business Logic

This project analyzes spatial-temporal tipping patterns using the NYC Yellow Taxi dataset. To compare tipping behavior while controlling for location, trip distance, and time, we calculate the Tipping Performance Index (TPI):

$$\text{TPI} = \text{Actual Tip \%} / \text{Baseline Tip \%}$$

The Baseline Tip Percentage is computed dynamically for cohorts defined by:

$$\text{Cohort} = (\text{Borough}, \text{Distance_Bucket}, \text{Hour_of_Day})$$

Trips are categorized into three distance buckets: *Short* (≤ 2.0 miles), *Medium* (2.0 to 8.0 miles), and *Long* (> 8.0 miles). TPI is a descriptive normalized indicator: values above 1 indicate tipping above the comparable cohort baseline; it does not establish a causal effect.

2. Cluster Configuration

The application was evaluated on an Apache Spark cluster deployed via Amazon EMR (release `emr-6.15.0`, Spark 3.4.1, Scala 2.12):

- Configured Master Node: 1 x `m4.large` (2 vCPUs, 8 GB RAM)
- Configured Worker Nodes: 2 x `m4.large` (4 vCPUs, 16 GB RAM total)
- Observed Spark Allocation: each supplied event log registered one executor with 4 Spark cores. Therefore, the logs verify shuffle processing, but not cross-executor network traffic.
- Storage: Amazon S3 stored raw inputs, outputs, and the application JAR.

3. Multi-Factor Query Optimization Implementation

We compared an Unoptimized Baseline Application and an Optimized Application. The result is an end-to-end comparison of several simultaneous changes, rather than an ablation study of one isolated factor.

1. Storage-Level Optimization (CSV vs. Parquet): The baseline reads 5.8 GB of row-oriented CSV, while the optimized application reads 1.5 GB of compressed Parquet. Spark can apply column pruning based on downstream references and supported predicate pushdown, reducing scan and parsing work.
2. Schema Resolution: `inferSchema=true` adds CSV parsing work, while Parquet stores schema metadata in its footer.

3. Double Broadcast Hints: The baseline disables automatic broadcasting and selects `SortMergeJoin`. The optimized application explicitly broadcasts the zone lookup and the aggregated baseline, converting both joins to `BroadcastHashJoin`. This removes almost all join-related shuffle. The aggregation and final ordering still require small shuffle stages.
4. Partition Tuning: `spark.sql.shuffle.partitions` is set to 32 instead of the default 200. This is a workload-specific experimental setting that reduces small-task overhead; it is not claimed as a universal optimum.
5. Output Path Difference: The baseline uses `coalesce(1)` and writes CSV, while the optimized application writes partitioned Parquet. This also contributes to the end-to-end runtime difference and limits per-factor attribution.

4. Quantitative Results

The supplied Spark event logs confirm substantial reductions in tasks, shuffle processing, and spill. The report runtime and event-log lifecycle use different timing boundaries, but both show a large speedup.

Metric	Unoptimized	Optimized	Impact
Join operators	SortMergeJoin	BroadcastHashJoin	Both joins changed
File format	CSV	Parquet	Lower scan/parsing work
Tasks	266	33	87.6% fewer
Input read	11,448 MB	1,187 MB	89.6% less
Shuffle read	1,101.8 MB	0.593 MB	Almost eliminated
Shuffle write	703.4 MB	0.417 MB	Almost eliminated
Memory spill	672 MB	0 MB	Eliminated
Report runtime	665 s	142 s	4.68x speedup
Event-log lifecycle	618.141 s	113.146 s	5.46x speedup

The event logs report zero remote shuffle read for both applications. Because each run registered only one executor, the approximately 1.1 GB baseline shuffle was processed locally on that executor. The experiment therefore demonstrates a reduction in shuffle I/O and sorting, but it does not directly measure cross-worker network savings.

5. Physical Execution Plan Analysis

- Unoptimized Plan: the final adaptive plan contains `SortMergeJoin`, `Exchange`, and `AQEShuffleRead` operators. It records approximately 1.1 GB of shuffle read, 703 MB of shuffle write, and 672 MB of memory spill.
- Optimized Plan: both joins appear as `BroadcastHashJoin`. Join-side repartitioning is removed, while the remaining aggregation and global sort account for approximately 0.59 MB of shuffle read and 0.42 MB of shuffle write. Executors, not the driver, process the large-table partitions.

6. Interpretation and Limitations

The supplied histories provide strong evidence that the optimized application performs less work: it scans less input, launches fewer tasks, nearly eliminates shuffle I/O, and avoids spill. They also confirm that the two intended join operators changed from `SortMergeJoin` to `BroadcastHashJoin`.

However, four limitations should be kept explicit:

- Only one executor was registered in each supplied event log, so cross-executor network reduction was not measured.
- Multiple variables changed simultaneously, including storage format, schema handling, join strategy, partition count, and output format.
- The two reported runtime pairs use different timing boundaries: external end-to-end timing versus Spark application lifecycle.
- A rigorous per-factor evaluation would require repeated runs and an ablation study with identical output semantics and format.

7. Conclusion

The optimized pipeline achieved a substantial and reproducible performance improvement. The project report records a 4.68x end-to-end speedup, while the Spark event logs show a 5.46x application-lifecycle speedup. The physical plans and metrics confirm that Parquet reduced scan work and that explicit broadcast hints replaced both `SortMergeJoins`, reducing shuffle read from approximately 1.1 GB to 0.59 MB and eliminating spill.

The main conclusion is therefore supported: storage format, join strategy, and partition sizing can greatly improve distributed data processing without adding hardware. The evidence should be described as a multi-factor end-to-end result and as a reduction in shuffle processing, rather than as proof of cross-worker network savings.